

Práctica – Tercera Aproximación.**Traductor de Expresiones a un Lenguaje en Notación Prefija**

Esta tercera parte de la práctica consiste en transformar el traductor de las sesiones previas. El primer traductor seguía un esquema conocido como traducción directa, es decir, la conversión e impresión de la entrada analizada se hace sobre la marcha al mismo tiempo que se explora el árbol de derivación. En la segunda aproximación se transformaba en un esquema de traducción más elaborado imprescindible para traducir a expresiones en notación prefija.

En la nueva propuesta ya nos centramos en la traducción a un lenguaje de programación que usa notación prefija. Ahora también nos interesa ser capaces de evaluar las expresiones, aunque lo haremos a través de un intérprete que procese la salida de nuestro traductor, con lo que tendríamos una cadena de evaluación automática.

Trabajo a realizar:

Para esta aproximación se pide la traducción de expresiones aritméticas y de asignaciones a variables de una notación infija como la que usamos habitualmente en el Lenguaje C a notación prefija que serán evaluadas por un intérprete de Lisp. En éste caso proponemos el uso de Árboles de Sintaxis Abstracta para realizar la traducción.

1. Leed el enunciado completo antes de comenzar.
2. En ésta práctica debéis adaptar el código de la sesión previa para cambiar la traducción con código diferido a otra con Árboles de Sintaxis Abstracta.
 - a. Leed el ejercicio de Árboles de Sintaxis Abstracta binarios.
 - b. Utilizad vuestro fichero de la segunda aproximación y copiado a **calc8.y**.
 - c. Incluid el código nuevo del ejercicio en vuestro fichero **calc8.y**.
 - d. La gramática debería ser la misma de **calc7.y** salvo que requiera alguna mejora. Revisad las especificaciones al final del enunciado.
 - e. Adaptad las acciones semánticas para crear un AST binario.
 - f. Incluid en la acción semántica de *Axioma* una llamada a la función **printAST2Prefix** para imprimir la traducción contenida en el AST.
3. Evaluad los resultados usando el intérprete online. Para ello:
 - a. Editad un fichero (por ejemplo, **prueba.txt**) con una serie de expresiones
 - b. Ejecutad **cat prueba.txt | ./calc8**
 - c. Copiad la salida y pegadla en la ventana del intérprete
 - d. con F8 se compila y ejecuta.
4. Alternativamente, instalad el intérprete de **clisp** de forma local usando
sudo apt-get install clisp
Probad vuestro programa usando:
Opción 1:
./calc8 <prueba.txt | clisp
Opción 2:
./calc8 <prueba.txt >prueba.l
clisp prueba.l

Breve Introducción al Lenguaje Lisp

Lisp es uno de los primeros Lenguajes de Programación desarrollados, un poco posterior a Fortran, que además sigue perviviendo en nuestros días. Estuvo además muy vinculado al desarrollo de la Inteligencia Artificial a partir de los años 60. Sus principales características son su enorme simplicidad de base y que su principal tipo de datos son las listas. Dado que un programa en Lisp se representa como una lista de listas de elementos, resulta factible procesar programas mediante otros programas. Es además el primer representante del paradigma de programación funcional y recursiva. Y también uno de los primeros lenguajes interpretados. Aunque ha caído en desuso, hay otras variantes como Scheme y Clojure que tienen bastante uso hoy en día.

Para esta práctica vamos a proponer una serie de variaciones iniciales. Disponéis de un intérprete online en https://rextester.com/l/common_lisp_online_compiler que os permitirá ir probando los ejemplos incluidos aquí. Con F8 se compila y ejecuta lo introducido en la ventana.

El tipo de datos genérico será una *Expresión*. Los casos elementales son:

1. *Número* (un literal de valor de tipo entero)
2. *Variable* (en principio para almacenar valores enteros)

Pero también tendremos elementos compuestos:

3. *Expresión* (Compuesta). Los elementos de una expresión serán *Operadores/Funciones*, y otras *Expresiones* (*Números*, *Variables* y *Expresiones Compuestas*). La notación para representar una *Expresión* Compuesta se basa en el uso de paréntesis:

<code>(* 2 3)</code>	para representar la multiplicación $2*3$
<code>(+ 1 a)</code>	para sumar el valor de una variable con un número
<code>(+ 1 (* 2 3))</code>	para representar una expresión más compleja $1+2*3$

Hay otros elementos que intervienen en el Lenguaje. Destacamos:

4. *Operador* o *Función*, en una expresión serán el primer elemento. Los demás elementos de la Expresión serán los parámetros de dicho Operador o Función.

Para asignar un valor a una variable usaremos la Función **setq**:

<code>(setq a 1)</code>	<code>a = 1</code>
<code>(setq a (+ 1 (* 2 3)))</code>	<code>a = 1+2*3</code>

No es necesario declarar las variables previamente. Usaremos nombres de variables de una única letra para empezar.

Para imprimir un valor usaremos la Función **print**

<code>(print a)</code>	imprimirá el valor de a
<code>(print (+ 1 (* 2 3)))</code>	imprimirá 7

Admite un único parámetro.

Para negar un valor podemos aplicar

<code>(- 0 a)</code>	que niega una variable
<code>(- 0 3)</code>	devuelve -3
<code>(- 0 (+ 1 (* 2 3)))</code>	devuelve -7

O más breve con un Operador menos unario.

<code>(- a)</code>	que niega una variable
<code>(- 3)</code>	devuelve -3
<code>(- (+ 1 (* 2 3)))</code>	devuelve -7

Más adelante necesitaremos definir funciones en LISP: para ello usaremos la Palabra Reservada **defun**. Incluimos algunos ejemplos:

```
(defun prueba ()  
  (print "hola"))
```

Define una función llamada **prueba** sin parámetros y cuyo cuerpo contiene una llamada a `(print "hola")`. Si a continuación evaluamos la Expresión:

`(prueba)` → Imprimirá "hola"

Podemos definir una función **prueba2** con un parámetro:

```
(defun prueba2 (p1)  
  (print p1))
```

Al evaluar:

`(prueba2 "adios")` → Imprimirá "adios"

Para definir una función que calcule el cuadrado de un valor:

```
(defun cuadrado (v)  
  (* v v))
```

`(print (cuadrado 4))` → Imprimirá 16

Lisp permite definir funciones recursivas. El factorial en versión recursiva sería:

```
(defun factorial (n)  
  (if (= n 0)  
      1  
      (* n (factorial (- n 1)))))  
)
```

`(print (factorial 5))` → Imprimirá 120

Un aspecto muy importante e interesante de Lisp es que cualquier expresión devuelve siempre algo, en nuestro caso serán los valores que se evalúan en la expresión. Tened en cuenta que *cualquier* expresión hace referencia a todas las expresiones de Lisp. Una secuencia de expresiones también se pueden considerar cómo una expresión en la que se devuelve el último valor evaluado. Ejemplos:

`(+ 1 2)` $\Rightarrow$ devuelve 3
`(setq A (+ 1 2))` $\Rightarrow$ asigna 3 a A y devuelve un 3
`(print (setq A (+ 1 2)))` $\Rightarrow$ imprime el 3 de `(setq A (+ 1 2))` y devuelve 3
`(print (print (setq A (+ 1 2))))` $\Rightarrow$ imprime dos veces un 3 y devuelve 3

Cuando se utiliza el intérprete de Lisp interactivo, es habitual que se imprima de forma adicional este valor final devuelto:

```
> A
3
> (+ 1 2)
3
```

[Como nota adicional, en el lenguaje C hay diferencias particulares respecto a Lisp, por ejemplo en el caso de la función **printf** que también devuelve un valor, pero en éste caso se trata de una indicación de si la impresión ha fallado y/o cuántos campos se han podido imprimir en caso contrario. El comportamiento no es comparable en ambos casos]

Esto nos servirá como introducción inicial para abordar la práctica.

Especificaciones

Proponemos una secuencia de pasos para abordar esta práctica. Incluimos muchas de la la aproximación previa cómo recordatorio:

1. Se pide adaptar el código de la tarea anterior (calc7) que utilizaba código diferido a un nuevo enfoque que utiliza Árboles de Sintaxis Abstracta binarios. Incluid las acciones semánticas apropiadas en las producciones para que se construya un AST durante el proceso de derivación. Los AST deberían transmitirse en el campo `$$node`. La traducción debe imprimirse en el axioma utilizando la función `printAST2Prefix`.
2. Prestad atención al diseño jerárquico de la gramática. Debería haber un *Axioma* que llama a una *Sentencia* que a su vez deriva en *Expresion* por un lado y en *Asignacion* por otro. La recursividad debe aplicarse en el *Axioma*. La traducción debería volcarse a la salida a nivel de *Axioma*. Para separar sentencias usaremos saltos de línea. Esto simplificará la gramática, reducirá potenciales conflictos y permitirá imprimir AST completos contruidos a nivel de *Sentencia*.

Comprobad que se cumplen las siguientes especificaciones ya planteadas en la práctica anterior.

3. La entrada `1+2*3` debería traducirse como `(+ 1 (* 2 3))`. Prestad atención a la precedencia y asociatividad definidas en la cabecera del fichero `calc7.y`.
4. En esta ocasión usaremos paréntesis para acotar las expresiones de salida.
5. Abordad la traducción de las *Variables*, tanto en la parte de componentes de una *Expresion*, como receptoras de una asignación.
6. Incluid una nueva sentencia para imprimir valores. En Lenguaje C debería ser `printf("%d",<expresion>)` pero nuestro Analizador Léxicografico no está adaptado para reconocer *Palabras Reservadas* (como `printf`), ni tratar la cadena de formato. Por ello proponemos una simplificación. La entrada `@ <expresion>` debe traducirse como `(print <expresion>)`.

Una vez hayáis comprobado que vuestro generador de AST binarios funciona correctamente podéis pasar a abordar el siguiente punto:

7. Intentad abordar la traductor de asignaciones encadenadas. Ejemplo:
`a=b=c=2+3`; debería traducirse como `(setq a (setq b (setq c (+ 2 3))))`

Entrega:

- Subid el fichero `calc8.y`.
- Incluid en la cabecera del fichero dos líneas de comentario iniciales. La primera con vuestros nombres y número de grupo. Y la segunda con los correos electrónicos (separados con un espacio).
- Entregad también un documento llamado `calc8.pdf` con una breve explicación del trabajo realizado.
- Antes de consolidar la entrega descargad todo para verificar que está todo y que funciona correctamente.